

Week 5

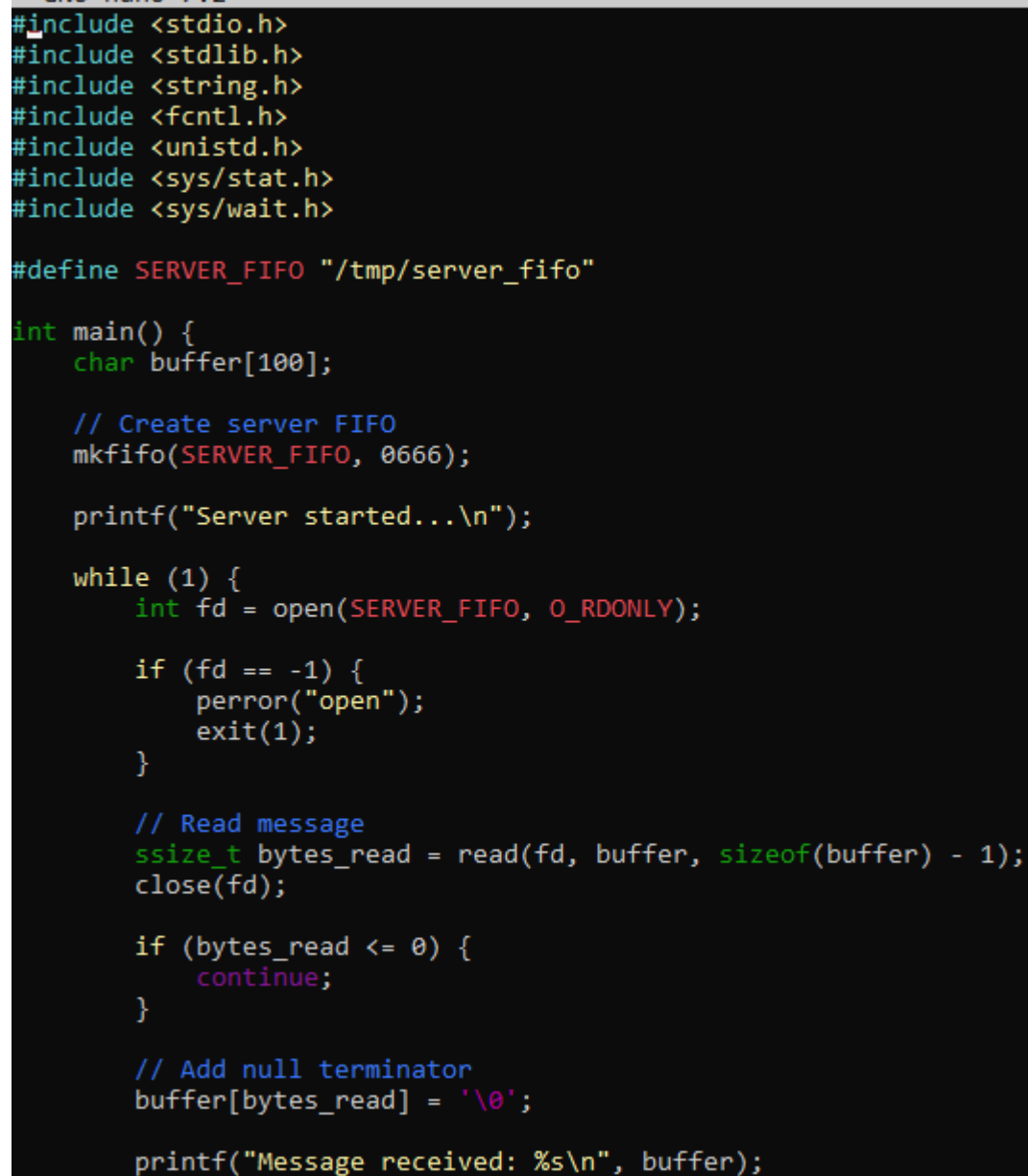
Task 1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.



```
LST@LAPTOP-EK5DCJVE ~$ nano server.c
LST@LAPTOP-EK5DCJVE ~$ |
```

Source code:



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

#define SERVER_FIFO "/tmp/server_fifo"

int main() {
    char buffer[100];

    // Create server FIFO
    mkfifo(SERVER_FIFO, 0666);

    printf("Server started...\n");

    while (1) {
        int fd = open(SERVER_FIFO, O_RDONLY);

        if (fd == -1) {
            perror("open");
            exit(1);
        }

        // Read message
        ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);
        close(fd);

        if (bytes_read <= 0) {
            continue;
        }

        // Add null terminator
        buffer[bytes_read] = '\0';

        printf("Message received: %s\n", buffer);
    }
}
```

```

// Create a child process to handle each client
if (fork() == 0) {
    char response[128];

    printf("Processing client request...\n");

    snprintf(response, sizeof(response),
             "Server received: %s", buffer);

    // Print response
    printf("Response: %s\n", response);

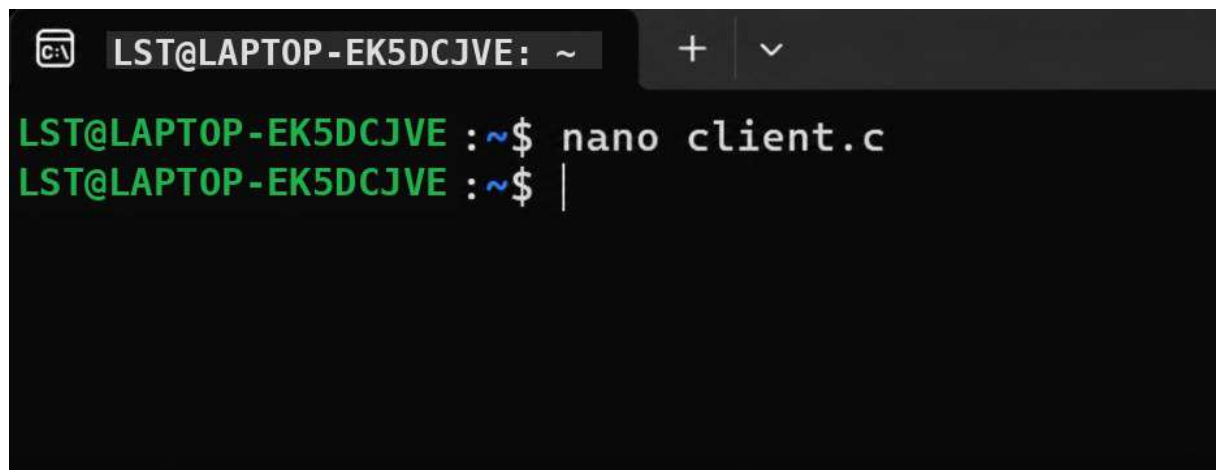
    exit(0);
}

// Check for completed child processes
waitpid(-1, NULL, WNOHANG);
}

unlink(SERVER_FIFO);

return 0;
}

```



A terminal window with a dark background. The title bar shows a file icon, the text "LST@LAPTOP-EK5DCJVE: ~", and window control buttons (+ and v). The terminal content shows a green prompt "LST@LAPTOP-EK5DCJVE :~\$" followed by the command "nano client.c". The next line shows the same prompt with a vertical cursor bar at the end.

```

LST@LAPTOP-EK5DCJVE: ~$ nano client.c
LST@LAPTOP-EK5DCJVE: ~$ |

```

```
#include <fcntl.h>
#include <unistd.h>

#define SERVER_FIFO "/tmp/server_fifo"

int main() {
    char message[100];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    int fd = open(SERVER_FIFO, O_WRONLY);

    if (fd == -1) {
        perror("open");
        exit(1);
    }

    write(fd, message, strlen(message) + 1);

    close(fd);

    printf("Message sent to server.\n");

    return 0;
}
```

Output:

```
LST@LAPTOP-EK5DCJVE: ~
LST@LAPTOP-EK5DCJVE: ~$ nano server.c
LST@LAPTOP-EK5DCJVE: ~$ gcc server.c -o server
LST@LAPTOP-EK5DCJVE: ~$ ./server
Server started on port 8080...
Waiting for client...
Client connected!
Received: Hello from client
Sent: Hello from server
|

LST@LAPTOP-EK5DCJVE: ~
LST@LAPTOP-EK5DCJVE: ~$ nano client.c
LST@LAPTOP-EK5DCJVE: ~$ gcc client.c -o client
LST@LAPTOP-EK5DCJVE: ~$ ./client
Enter message: Hello from client
Received from server: Hello from server
LST@LAPTOP-EK5DCJVE: ~$ |
```

Task2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Source code:

```
LST@LAPTOP-EK5DCJVE~$ nano signal_demo.c
LST@LAPTOP-EK5DCJVE~$ |
```

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {
    if (sig == SIGINT) {
        printf("\nSIGINT received!\n");
        printf("This is usually generated by pressing Ctrl+C.\n");
    }

    else if (sig == SIGTERM) {
        printf("\nSIGTERM received!\n");
        printf("Termination request received.\n");
    }

    else if (sig == SIGUSR1) {
        printf("\nSIGUSR1 received!\n");
        printf("User-defined signal received.\n");
    }
}

int main() {
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1) {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}

```

[illegible]